MLOps PROJECT REPORT

Curriculum-Industry Skill Feature Store Using Feast

Feature Engineering • Feast • Historical Retrieval • Online Store • Machine Learning

STUDENT DETAILS

Name	P.HEMANJANEYULU
Register Number	231FA04925
Section	15

MLOps • Feast • Feature Engineering • Machine Learning

1. Executive Summary

This project converts a 1,000-student curriculum-industry skill-gap dataset into a reusable Feast feature store and connects the feature store to a machine-learning workflow. The implementation demonstrates data cleaning, feature engineering, entity creation, a Feast FileSource and FeatureView, historical feature retrieval, online materialization into SQLite, online feature retrieval, and a Random Forest regression model for predicting Skill_Gap_Index.

The core MLOps idea is feature consistency: the same centrally defined feature set is reused during model training and prediction rather than duplicating transformation logic in separate scripts. The assignment explicitly requires demonstration of feature engineering, Feast entity/data source/FeatureView creation, feast apply, historical retrieval, materialization, online retrieval, machine-learning usage, and GitHub documentation.

Reference results from the supplied project: MAE 4.3342 | RMSE 5.7924 | R^2 0.7827 | Train/Test 800/200 | CSE_0001 prediction 8.30

Project Deliverables Covered

- Cleaned and engineered student features
- Feast entity: student_id
- Feast FileSource and FeatureView: student_skill_features
- Historical retrieval via get_historical_features()
- Materialization to SQLite online store
- Online retrieval via get_online_features()
- Random Forest model for Skill_Gap_Index
- Required README analysis and implementation guidance

The report intentionally preserves the project terminology used in the supplied assignment and README. Where the dataset or project does not support a real-world claim, the report identifies the information as synthetic, reference, or project-generated rather than presenting it as external evidence.

2. Table of Contents

Section	Location
3. Assignment Alignment	Page 4
4. Dataset Profile	Page 5
5. Data Cleaning and Feature Engineering	Page 6
6. Feast Architecture	Page 7
7. Feast Implementation	Page 8
8. Historical and Online Retrieval	Page 9
9. Machine-Learning Model and Results	Page 10
10. Required Analysis Questions	Page 11
11. Run and Submission Guide	Page 13
12. Limitations and Future Improvements	Page 14

Assignment requirements used as the report backbone: feature engineering, Feast entity, data source, FeatureView, feast apply, historical retrieval, materialization, online retrieval, ML integration, and GitHub documentation.

3. Assignment Alignment

The supplied assignment asks the student to use the curriculum-industry skill-gap dataset and convert it into a simple Feast-based feature store. It specifically requires ten implementation demonstrations and a README containing eleven analysis responses.

Req.	Assignment requirement	Project evidence
1	Feature engineering	Implemented in src/prepare_data.py; raw scores are converted into reusable model-ready features.
2	Feast entity	student_id uniquely identifies a student.
3	Feast data source	FileSource points to processed student feature data with event_timestamp.
4	FeatureView	student_skill_features stores reusable academic, technical, soft-skill and industry features.
5	feast apply	Registers or updates the Feast objects from feature_repo.
6	Historical retrieval	get_historical_features() creates point-in-time training features.
7	Materialization	Loads feature values into the SQLite online store.
8	Online retrieval	get_online_features() retrieves the current feature vector for a student.
9	ML usage	RandomForestRegressor predicts Skill_Gap_Index from Feast features.
10	Documentation	README, architecture diagram, setup guide, outputs, tests, and this report are included in the project package.

4. Dataset Profile

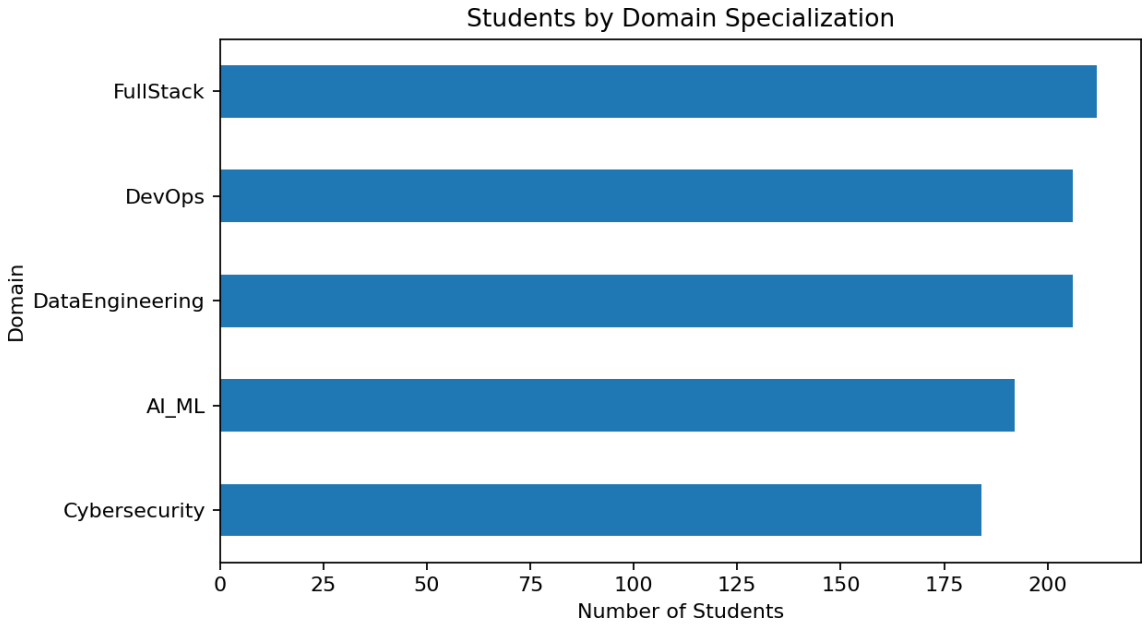
The supplied CSV contains 1,000 student records and 20 original columns. The project README identifies the dataset as synthetic/random and spanning academic, technical, practical-experience, soft-skill, and industry-readiness variables.

Item	Value
Rows	1,000
Original columns	20
Target	Skill_Gap_Index
Entity	student_id derived from Student_ID
Domain values	AI_ML, Cybersecurity, DataEngineering, DevOps, FullStack
Numeric columns	18
Categorical columns	2

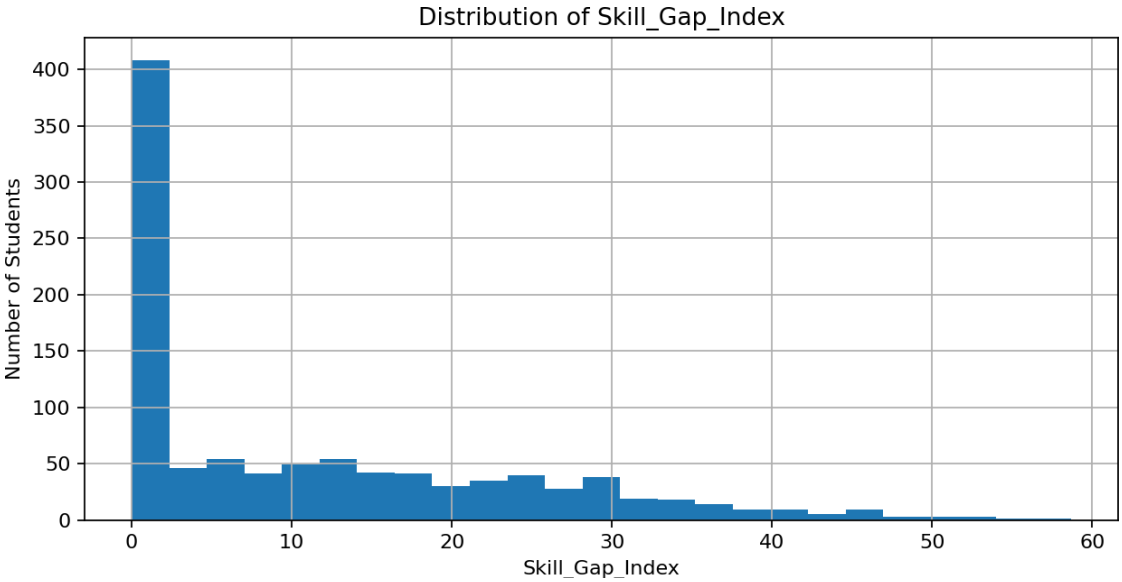
Original Dataset Columns

Student_ID, CGPA, DSA_Score, System_Design_Score, DBMS_Score, OOP_Proficiency, Cloud_DevOps_Score, Web_Dev_Score, Problem_Solving_Rating, Aptitude_Score, Communication_Rating, Teamwork_Score, Github_Projects_Count, Internship_Duration_Months, Certifications_Count, Hackathons_Participated, Mock_Interview_Score, Domain_Specialization, Curriculum_Updated_Year, Skill_Gap_Index

Domain Mix



Target Distribution


Distribution of Skill_Gap_Index
Number of Students
Skill_Gap_Index
0
50
100
150
200
250
300
350
400
0
10
20
30
40
50
60

5. Data Cleaning and Feature Engineering

Data Cleaning Pipeline

Step	Operation	Purpose
1	Read supplied CSV	Load the raw skill-gap dataset using pandas.
2	Remove duplicate student IDs	Keep one record per Student_ID.
3	Normalize numeric fields	Convert numeric columns to numeric types.
4	Handle missing numeric values	Fill missing numeric values with the median.
5	Handle missing categorical values	Fill missing categorical values with the mode.
6	Create event_timestamp	Provide a timestamp required for Feast point-in-time retrieval.
7	Separate target	Keep Skill_Gap_Index out of the FeatureView to avoid target leakage.

Engineered Feast Features

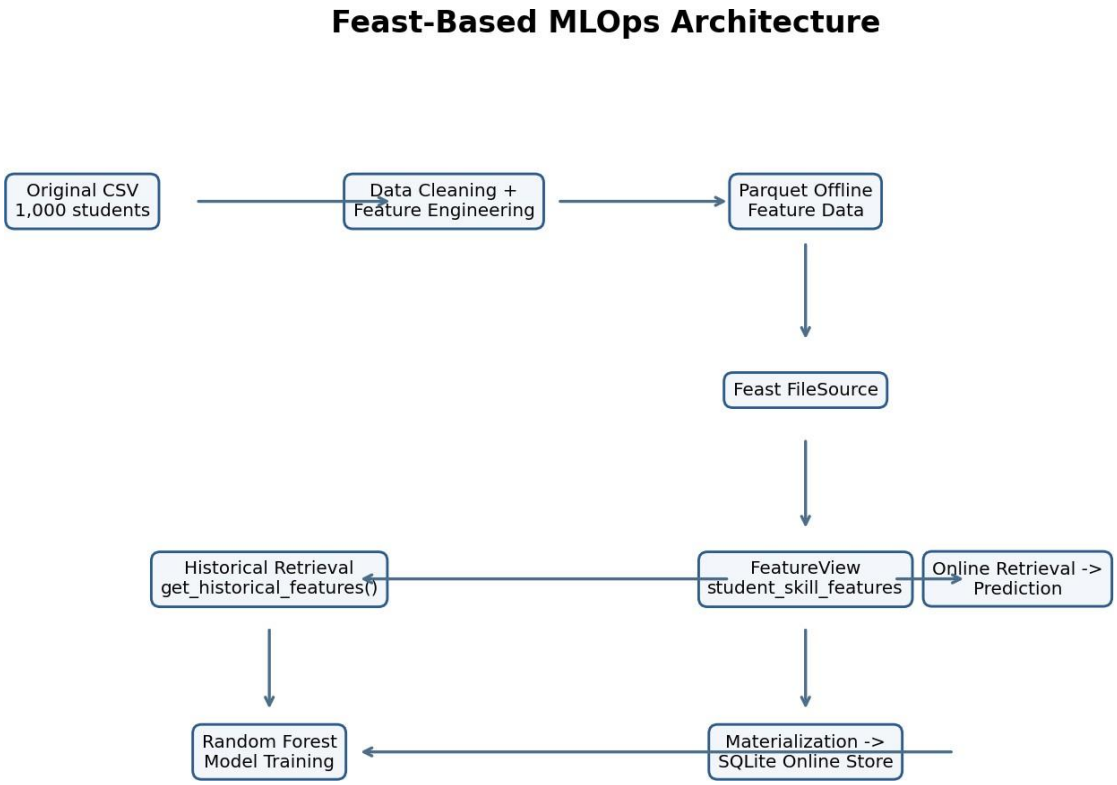
Feature	Meaning / source	Feature	Meaning / source
cgpa	Academic CGPA	dsa_score	DSA score
system_design_score	System design score	dbms_score	DBMS score
oop_proficiency	OOP proficiency	cloud_devops_score	Cloud/DevOps score
web_dev_score	Web development score	problem_solving_score	Problem-solving score normalized to approx. 0-100
aptitude_score	Aptitude score	communication_score	Communication rating scaled to approx. 0-100
teamwork_score	Teamwork score	github_projects_count	GitHub project count
internship_duration_months	Internship duration	certifications_count	Certification count
hackathons_participated	Hackathon participation	mock_interview_score	Mock interview score
curriculum_updated_year	Curriculum update year	domain_code	Encoded specialization
technical_skill_mean	Mean of six technical skills	soft_skill_mean	Mean soft-skill indicator
industry_exposure_score	Industry exposure composite	practical_experience_score	Practical experience composite
curriculum_age_years	Age of curriculum year relative to project snapshot		

Example Feature Calculation

```
technical_skill_mean = mean(DSA, System Design, DBMS, OOP, Cloud/DevOps, Web Development)
problem_solving_score = Problem Solving Rating / 25 communication_score = Communication Rating * 20
```

Target leakage prevention: Skill_Gap_Index remains outside the FeatureView because it is the supervised-learning target, not an input feature.

6. Feast Architecture



The architecture separates raw data preparation from feature serving. The processed feature data is the offline source for Feast. Historical retrieval supports point-in-time model training, while materialization moves the latest values into the SQLite online store for low-latency retrieval at prediction time.

Offline vs Online Store

Component	Purpose in this project
Offline store	Holds historical feature data and supports point-in-time training retrieval.
Online store	Holds the latest feature values for quick online retrieval during prediction.
Feast registry	Records the definitions of entities, data sources, and FeatureViews.

7. Feast Implementation

Entity

```
student = Entity( name="student_id", join_keys=["student_id"], description="Unique student identifier." )
```

The entity is student_id. It is the stable join key used to retrieve the correct feature vector for each student.

FileSource

```
student_source = FileSource( name="student_skill_features_source",
path="../data/processed/student_features.parquet", timestamp_field="event_timestamp" )
```

The FileSource uses the processed Parquet feature file and event_timestamp to support historical, point-in-time retrieval.

FeatureView

```
student_skill_features = FeatureView( name="student_skill_features", entities=[student],
ttl=timedelta(days=3650), online=True, source=student_source )
```

The FeatureView contains 23 source/engineered feature columns plus the entity key; the target Skill_Gap_Index is deliberately excluded. The project README describes this collection as the reusable academic, technical, soft-skill, practical-experience and curriculum feature set.

Registration with feast apply

```
cd feature_repo feast apply
```

The assignment asks for feast apply to register or update the configured Feast objects in the feature repository. After registration, the project can retrieve historical features and materialize online features.

8. Historical and Online Retrieval

Historical Retrieval

```
store.get_historical_features( entity_df=entity_df, features=["student_skill_features:cgpa", ...] )
```

Historical retrieval returns the feature values that were valid for each entity at the relevant event timestamp. In this project it provides the point-in-time feature set used for model training.

Online Materialization

```
store.materialize( start_date=..., end_date=... )
```

Materialization copies the latest feature values from the offline source into the SQLite online store for the requested time range.

Online Retrieval Sample - CSE_0001

Feature	Online value
student_id	CSE_0001
cgpa	6.87
dsa_score	46.0
mock_interview_score	86.0
domain_code	3.0
technical_skill_mean	39.0
soft_skill_mean	56.907
industry_exposure_score	51.0
practical_experience_score	48.0
curriculum_age_years	3.0

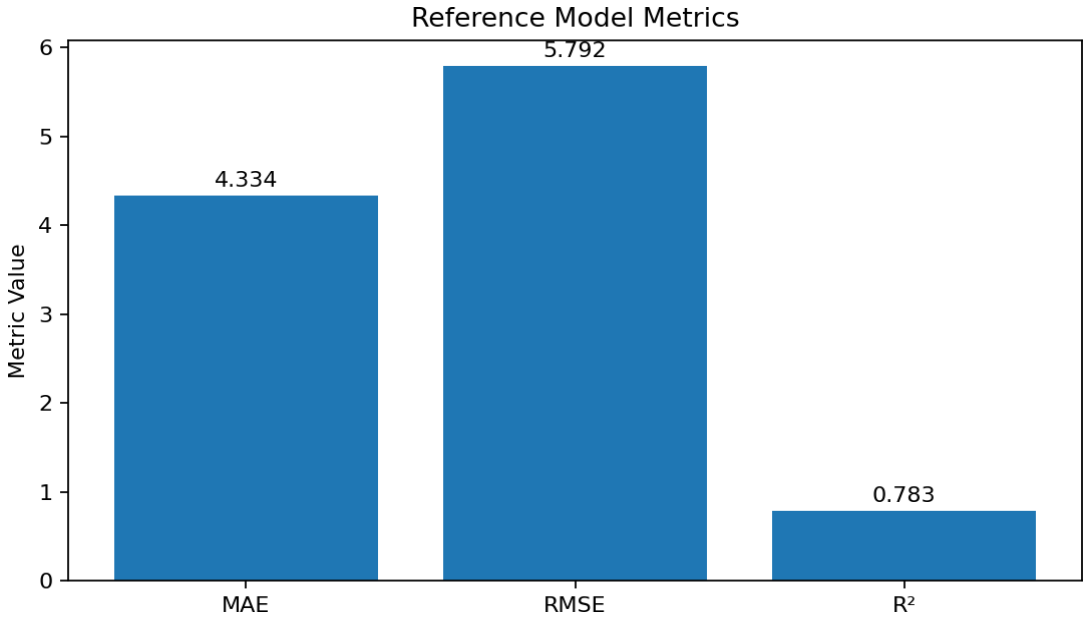
The project output file records the complete online feature vector. The values above are selected to show the most important identity, skill, experience, composite, and curriculum features.

Prediction Output

Student: CSE_0001 | Predicted Skill_Gap_Index: 8.30

9. Machine-Learning Model and Results

The project trains a RandomForestRegressor using the Feast-derived feature columns. The README specifies a 20% test split with random_state=42. The reported reference output contains 800 training rows and 200 test rows.



Metric	Value	Interpretation
MAE	4.3342	Average absolute prediction error on the reference test set.
RMSE	5.7924	Penalizes larger prediction errors more strongly than MAE.
R^2	0.7827	Explained variance indicator for the reference test set.

Reference Training Configuration

Item	Configuration
Algorithm	RandomForestRegressor
Target	Skill_Gap_Index
Train rows	800
Test rows	200
Split	80% train / 20% test, random_state=42

The README labels these metrics as reference results. They are useful as a reproducibility baseline; the local Feast workflow should still be executed after installing the project dependencies.

10. Required Analysis Questions

1. What is the entity in your Feast implementation?

The entity is student_id. It uniquely identifies one student and is used as the join key for feature retrieval.

2. List the features stored in your FeatureView.

The FeatureView stores cgpa, dsa_score, system_design_score, dbms_score, oop_proficiency, cloud_devops_score, web_dev_score, problem_solving_score, aptitude_score, communication_score, teamwork_score, github_projects_count, internship_duration_months, certifications_count, hackathons_participated, mock_interview_score, curriculum_updated_year, domain_code, technical_skill_mean, soft_skill_mean, industry_exposure_score, practical_experience_score, and curriculum_age_years.

3. Explain how one feature was calculated.

technical_skill_mean is the arithmetic mean of DSA, System Design, DBMS, OOP, Cloud/DevOps, and Web Development scores.

4. What is the difference between your original dataset and the feature dataset?

The original dataset contains raw student attributes and the target. The feature dataset contains cleaned, model-ready numerical features, a Feast entity key, and an event timestamp. The target is kept separately for supervised learning and is not registered as an input feature.

5. What is the purpose of the offline store?

It keeps historical feature data and supports point-in-time retrieval for training and analysis.

6. What is the purpose of the online store?

It keeps the latest feature values so applications can retrieve features quickly for prediction.

7. What is the purpose of feast apply?

It registers or updates the configured Feast entities, data sources, and FeatureViews.

8. What does materialization do?

It loads feature values from the offline source into the online store for the specified time range.

9. What is the advantage of retrieving features through Feast instead of manually calculating them separately during training and prediction?

The same centrally defined feature transformations can be reused for training and inference, reducing duplicated logic and improving training-serving consistency.

10. State two limitations of your current dataset.

(1) The dataset is synthetic/random and may not represent real hiring or curriculum evidence. (2) It is a single snapshot rather than a rich longitudinal record of how a student's skills evolve over time.

11. State two ways your feature store could be improved when more curriculum and industry evidence becomes available.

(1) Add time-versioned curriculum outcomes, placement outcomes, job descriptions, interview results, and industry skill-demand evidence. (2) Add richer domain-specific features plus production monitoring such as feature freshness, drift detection, data-quality checks, and model-performance tracking.

11. Run and Submission Guide

Environment Setup

```
python -m venv .venv # Windows .venv\Scripts\activate # Linux/macOS source .venv/bin/activate pip install -r requirements.txt
```

Execution Sequence

Step	Action	Command
01	Prepare data	`python src/prepare_data.py`
02	Register Feast objects	`cd feature_repo -> feast apply -> cd ..`
03	Historical retrieval	`python src/get_historical.py`
04	Train model	`python src/train_model.py`
05	Materialize online store	`python src/materialize.py`
06	Online retrieval	`python src/get_online.py`
07	Final prediction	`python src/predict.py`

Repository Structure

```
MLOps-Feast-SkillGap/ ■■■ README.md ■■■ requirements.txt ■■■ feature_repo/ ■ ■■■ feature_store.yaml ■ ■■■ features.py ■■■ data/ ■ ■■■ raw/ ■ ■■■ processed/ ■■■ src/ ■ ■■■ prepare_data.py ■ ■■■ get_historical.py ■ ■■■ train_model.py ■ ■■■ materialize.py ■ ■■■ get_online.py ■ ■■■ predict.py ■■■ outputs/ ■■■ models/ ■■■ tests/ ■■■ notebooks/
```

GitHub Submission

The assignment requires the repository name format <231FA04925>MLOps-Feast-SkillGap and asks for the GitHub repository link to be submitted through the faculty Google Form. Before submission, replace the README placeholders for register number and section.

12. Limitations and Future Improvements

Current Dataset Limitations

• Synthetic/random records may not fully reflect real student outcomes, job-market requirements, or employer assessment processes.

• The dataset is primarily a single snapshot, so it provides limited evidence about skill progression, curriculum changes over time, or outcome trajectories.

Feature Store Improvements

• Introduce time-versioned industry skill-demand evidence, curriculum outcomes, internship outcomes, placement results, job descriptions, and interview signals.

• Add richer domain-specific feature groups for AI/ML, Cybersecurity, Data Engineering, DevOps, and Full Stack pathways.

• Add data-quality validation, feature freshness monitoring, feature drift detection, model monitoring, and alerting.

• Move from local FileSource + SQLite development to a production-grade offline/online store architecture when deployment scale requires it.

MLOps Maturity Path

Stage	Next-step capability
Level 1 - Repeatable	Manual but scripted data preparation, feature engineering, and model training.
Level 2 - Feature Reuse	Central Feast definitions reused for historical and online retrieval.
Level 3 - Production Monitoring	Add data quality, feature freshness, drift, and model monitoring.
Level 4 - Automated Delivery	CI/CD, automated training pipelines, registry promotion, and rollback.
Level 5 - Continuous Learning	Scheduled or event-driven retraining using new curriculum and industry evidence.

Final outcome: a student-skill feature pipeline that can support consistent historical training and online prediction while providing a clear path from classroom assignment to production-style MLOps practice.

Appendix A. Key Project Files

File	Purpose
README.md	Assignment-aligned documentation, analysis answers, setup and submission instructions.
feature_repo/features.py	Entity, FileSource and FeatureView definitions.
feature_repo/feature_store.yaml	Feast repository configuration.
src/prepare_data.py	Data cleaning and feature engineering.
src/get_historical.py	Historical feature retrieval.
src/train_model.py	Random Forest training and metrics.
src/materialize.py	Online store materialization.
src/get_online.py	Online feature retrieval.
src/predict.py	Feature retrieval + final prediction.
outputs/model_metrics.json	Reference model metrics.
outputs/online_features_sample.json	Online feature retrieval example.
outputs/historical_features_sample.csv	Historical feature output sample.
models/skill_gap_model.joblib	Trained reference model artifact.
tests/test_prepare_data.py	Basic data-preparation test coverage.